

Programming Techniques 2026–2027

Lecture 3: Basic I/O and Arrays

Hannu Parviainen

Universidad de la Laguna

October 5, 2026

Lecture 2 Recap

Recap: declarations

- ▶ Five intrinsic types: integer, real, complex, logical, and character.
- ▶ Always write `implicit none`, right after any use statements.
- ▶ Declarations come before the executable statements:
`type[, attributes] :: names [= value]`
- ▶ A variable can be given an initial value in its declaration.
- ▶ The parameter attribute makes a **named constant**. Its value is fixed at compile time, and it can be used in later declarations.

```
program declarations
  implicit none
  real, parameter :: pi = 3.14159
  real, parameter :: two_pi = 2 * pi
  integer :: n_planets = 8
  real :: mass, radius = 1.0
  logical :: converged = .false.
  character(len=12) :: star_name = 'Vega'
  complex :: visibility
end program declarations
```

Recap: expressions and assignment

- ▶ Arithmetic: `**`, `*`, `/`, `+`, `-`
- ▶ Relational: `<`, `<=`, `==`, `/=`, `>=`, `>`
“Not equal” is `/=`, because `!` starts a comment.
- ▶ Logical: `.not.`, `.and.`, `.or.`, `.eqv.`, `.neqv.`
- ▶ Assignment evaluates the right-hand side first, then silently converts it to the type of the left-hand side.
- ▶ Do not compare reals with `==`, but compare within a tolerance instead.

```
program expressions
  implicit none
  integer :: n_stars
  real :: m1 = 1.0, m2 = 1.00001
  logical :: equal_mass
  n_stars = 3.7 ! truncated to 3, -Wall warns
  equal_mass = abs(m1 - m2) < 1.0e-4
  print *, n_stars, equal_mass, m1 /= m2
end program expressions
```

Recap: conditional execution

- ▶ One-line if: runs a single statement when the condition is true.
- ▶ Block if: else if and else are both optional.
- ▶ select case chooses between single values, such as (0), and ranges, such as (2:7) or (2:). An optional case default catches everything else.

```
program conditionals
  implicit none
  real :: e = 0.3
  integer :: n_planets = 3
  if (e < 0.0) print *, 'Invalid eccentricity'
  if (e < 1.0) then
    print *, 'Elliptical orbit (bound)'
  else if (e > 1.0) then
    print *, 'Hyperbolic orbit (unbound)'
  else
    print *, 'Parabolic orbit'
  end if
  select case (n_planets)
  case (0)
    print *, 'No known planets'
  case (2:)
    print *, 'Multi-planet system'
  end select
end program conditionals
```

Recap: loops

- ▶ `do i = start, stop [, step]` runs a fixed number of times. The step defaults to 1.
- ▶ `do while (condition)` repeats while the condition is true.
- ▶ A plain `do` repeats until an `exit`.
- ▶ `exit` leaves the loop, and `cycle` skips to its next iteration.
- ▶ Loops can be named, and `exit` and `cycle` can name the loop they act on. Otherwise they act on the innermost loop.

```

program loops
  implicit none
  integer :: i, frame, total
  total = 0
  do i = 1, 9, 2                ! i = 1, 3, 5, 7, 9
    total = total + i
  end do
  do while (total > 1)
    total = total / 2
  end do
  frame = 0
  frames: do
    frame = frame + 1
    if (frame > 100) exit frames
    if (frame >= 50 .and. frame <= 59) cycle frames
    print *, 'Reducing frame', frame
  end do frames
end program loops

```

Basic Input and Output

Output: print and write

- ▶ `print *`, `items` and `write(*,*)` items both write a line to standard output, normally the terminal.
- ▶ In `write(*,*)`, the first `*` is the unit (standard output) and the second is the format. A `*` format lets the compiler choose the layout (*list-directed* output).
- ▶ Each statement writes one line, and `print *` alone writes an empty line.
- ▶ Formats for controlling the layout come later in the course.

```
program output
  implicit none
  real :: radius = 6371.0
  integer :: n_moons = 1
  print *, 'Earth radius:', radius, 'km'
  write(*,*) 'Moons:', n_moons, 'twice:', 2*n_moons
  print *
  print *, 'Is Earth a planet?', .true.
end program output
```

```
$ gfortran output.f90 && ./a.out
Earth radius:   6371.00000          km
Moons:          1 twice:           2

Is Earth a planet? T
```

Input: read

- ▶ `read *`, `items` and `read(*,*)`
items read from standard input,
normally the keyboard.
- ▶ The values can be separated by
spaces, commas, or line breaks. If a
line runs out, `read` continues on the
next one.
- ▶ Each `read` starts on a new line, and
values left over on the current line are
skipped.
- ▶ Invalid input, such as text where a
number is expected, stops the
program with a runtime error.

```
program density
  implicit none
  real, parameter :: pi = 3.14159
  real :: mass, radius, rho
  print *, 'Mass (kg) and radius (m)?'
  read *, mass, radius
  rho = mass / (4.0/3.0 * pi * radius**3)
  print *, 'Mean density (kg/m^3):', rho
end program density
```

```
$ gfortran density.f90 && ./a.out
Mass (kg) and radius (m)?
5.97e24 6.371e6
Mean density (kg/m^3):    5511.41650
```

Reading until the end of input

- ▶ With `iostat=ios`, `read` reports a status instead of stopping: 0 is success, negative is the end of the input, and positive is an error.
- ▶ This lets a program read any number of values.
- ▶ The shell can connect standard input and output to files or other programs, with no change to the code:
`./a.out < mags.txt,`
`./a.out > out.txt, or`
`prog1 | ./a.out.`

```
program mean
  implicit none
  real :: x, total = 0.0
  integer :: n = 0, ios
  do
    read(*, *, iostat=ios) x
    if (ios > 0) print *, 'Not a number, stopping'
    if (ios /= 0) exit
    total = total + x
    n = n + 1
  end do
  if (n > 0) print *, n, 'values, mean =', total / n
end program mean
```

```
$ ./a.out < mags.txt      # mags.txt: 4.2, 3.9, 4.5
                        3 values, mean =    4.20000029
```

Catching input errors

- ▶ `iomsg=msg` gives the error message.
- ▶ After a failed read, the input position is undefined. So read the whole line as text ('(a)'), then convert it with an *internal read* from the string.
- ▶ `read` does not check that values make sense. Test ranges yourself.

```
Give a number:
abc
Bad real number in item 1 of list input
Give a number:
2.5
Got 2.50000000
```

```
program read_number
  implicit none
  real :: x
  integer :: ios
  character(len=100) :: line
  character(len=200) :: msg
  do
    print *, 'Give a number:'
    read(*, '(a)', iostat=ios) line
    if (ios /= 0) stop 'No input'
    read(line, *, iostat=ios, iomsg=msg) x
    if (ios == 0) exit
    print *, trim(msg)
  end do
  print *, 'Got', x
end program read_number
```

Arrays

Arrays in Fortran

Idea

Arrays (vectors, matrices, tensors) hold multiple values of the same type. Elements are accessed by subscripts.

Examples (declarations)

```
real, dimension(6) :: X
```

```
real, dimension(1:5,1:3) :: Y
```

Every array has

Type (e.g., real), a rank (number of dimensions), bounds for each dimension, and a value for each element.

 $X =$

X(1)	X(2)	X(3)	X(4)	X(5)	X(6)
------	------	------	------	------	------

 $Y =$

Y(1,1)	Y(1,2)	Y(1,3)
Y(2,1)	Y(2,2)	Y(2,3)
Y(3,1)	Y(3,2)	Y(3,3)
Y(4,1)	Y(4,2)	Y(4,3)
Y(5,1)	Y(5,2)	Y(5,3)

Array Terminology

Definitions

- ▶ **Rank:** number of dimensions (e.g., 1D, 2D).
- ▶ **Bounds:** lower/upper index limits in each dimension.
- ▶ **Extent:** number of elements along a dimension.
- ▶ **Size:** total number of elements.
- ▶ **Shape:** tuple of extents.
- ▶ **Conformable:** same shape (element-wise operations valid).

Explicit-Shape Declarations

Forms

- ▶ `real, dimension(100) :: R`
- ▶ `real, dimension(1:10,1:10) :: S`
- ▶ `real :: T(10,10)`
- ▶ `real, dimension(-10:-1) :: X`
- ▶ `integer, parameter :: lda = 5`
`real, dimension(0:lda-1) :: Y`
- ▶ `real, dimension(1+lda*lda,10) :: Z`

Notes

Default lower bound is 1; bounds may begin/end anywhere; arrays can be zero-sized if bounds imply size 0.

Conformance Rules

Element-wise operations

Arrays (or sections) in an expression or assignment must conform: they must have the same shape. A scalar conforms with any array.

Examples

```
real :: A(3,4), B(4,3), C(3,4), D(3,4)
C = 1.0    ! valid: the scalar is copied to every element
C = D      ! valid: same shape, [3,4]
B = A      ! invalid: same size (12), but different shape
```

Checking

Here the shapes are known when compiling, so gfortran rejects the last line. When shapes are only known at run time, compile with `-fcheck=all`.

Array Element Order

- ▶ Fortran uses **column-major** order: the first index varies fastest, so the elements of each column are next to each other in memory.
- ▶ The same order is used by `print`, `read`, and `reshape`.
- ▶ C and NumPy use the opposite, *row-major*, order by default.

real :: A(3,4)

A(1,1)	A(1,2)	A(1,3)	A(1,4)
A(2,1)	A(2,2)	A(2,3)	A(2,4)
A(3,1)	A(3,2)	A(3,3)	A(3,4)

memory:

A(1,1)	A(2,1)	A(3,1)	A(1,2)	A(2,2)	A(3,2)	A(1,3)	A(2,3)	A(3,3)	A(1,4)	A(2,4)	A(3,4)
1	2	3	4	5	6	7	8	9	10	11	12

Loop Order Matters

- ▶ The CPU reads memory in blocks (cache lines), so a loop is fastest when consecutive iterations use neighbouring elements.
- ▶ In Fortran, put the **first** index in the **innermost** loop.
- ▶ Compiled with -O2, the row-by-row loop here is about three times slower, and the gap grows with the array size.
- ▶ In C and NumPy, it is the other way round.

```
program loop_order
  implicit none
  integer, parameter :: n = 5000
  integer :: a(n,n), s1 = 0, s2 = 0, i, j
  real :: t0, t1, t2
  a = 1
  call cpu_time(t0)
  do j = 1, n          ! fast: first index in the inner loop
    do i = 1, n
      s1 = s1 + a(i,j)
    end do
  end do
  call cpu_time(t1)
  do i = 1, n          ! slow: each step jumps n elements
    do j = 1, n
      s2 = s2 + a(i,j)
    end do
  end do
  call cpu_time(t2)
  print *, t1 - t0, t2 - t1, s1 == s2 ! ~0.005 s, ~0.015 s, T
end program loop_order
```

Subscript Triplets

General form

[lb] : [ub] [:stride] with scalar integer expressions.

Examples

```
A(:)      ! whole array
A(3:9)    ! A(3) .. A(9) step 1
A(3:9:1)   ! same as above
A(m:n)    ! lower/upper bounds
A(m:n:k)  ! step k
A(8:3:-1)  ! descending section
A(8:3)     ! zero-sized (default step 1)
A(m:)     ! m .. upper bound
A(:n)     ! lower bound .. n
A(::2)     ! every 2nd element
A(m:m)    ! 1-element section
A(m)      ! scalar element (not a section)
```

Array Indexing: Single Element

Indexing Example

`A(3,4)`

Selects one element at row 3, column 4.

Declaration

```
real :: A(5,5)
```

Result

A scalar.

$A =$

A(1,1)	A(1,2)	A(1,3)	A(1,4)	A(1,5)
A(2,1)	A(2,2)	A(2,3)	A(2,4)	A(2,5)
A(3,1)	A(3,2)	A(3,3)	A(3,4)	A(3,5)
A(4,1)	A(4,2)	A(4,3)	A(4,4)	A(4,5)
A(5,1)	A(5,2)	A(5,3)	A(5,4)	A(5,5)

Array Indexing: Row Section

Indexing Example

`A(2,:)`

Selects all columns on row 2.

Declaration

```
real :: A(5,5)
```

Result

A rank-1 array of size 5. The scalar subscript removes a dimension: `A(2:2,:)` would keep it, with shape `[1,5]`.

$A =$

A(1,1)	A(1,2)	A(1,3)	A(1,4)	A(1,5)
A(2,1)	A(2,2)	A(2,3)	A(2,4)	A(2,5)
A(3,1)	A(3,2)	A(3,3)	A(3,4)	A(3,5)
A(4,1)	A(4,2)	A(4,3)	A(4,4)	A(4,5)
A(5,1)	A(5,2)	A(5,3)	A(5,4)	A(5,5)

Array Indexing: Column Section

Indexing Example

`A(:,3)`

Selects all rows in column 3.

Declaration

`real :: A(5,5)`

Result

A rank-1 array of size 5.

$A =$

A(1,1)	A(1,2)	A(1,3)	A(1,4)	A(1,5)
A(2,1)	A(2,2)	A(2,3)	A(2,4)	A(2,5)
A(3,1)	A(3,2)	A(3,3)	A(3,4)	A(3,5)
A(4,1)	A(4,2)	A(4,3)	A(4,4)	A(4,5)
A(5,1)	A(5,2)	A(5,3)	A(5,4)	A(5,5)

Array Indexing: Subarray

Indexing Example

`A(2:4, 2:5)`

Selects rows 2–4 and columns 2–5 (a 3x4 block).

Declaration

```
real :: A(5,5)
```

Result

A rank-2 array of shape [3,4].

$A =$

A(1,1)	A(1,2)	A(1,3)	A(1,4)	A(1,5)
A(2,1)	A(2,2)	A(2,3)	A(2,4)	A(2,5)
A(3,1)	A(3,2)	A(3,3)	A(3,4)	A(3,5)
A(4,1)	A(4,2)	A(4,3)	A(4,4)	A(4,5)
A(5,1)	A(5,2)	A(5,3)	A(5,4)	A(5,5)

Array Indexing: Strided Selection

Indexing Example

`A(1:5:2, 2:5:2)`

Selects odd rows and even columns.

Declaration

`real :: A(5,5)`

Result

A rank-2 array of shape `[3,2]`: rows 1, 3, 5 and columns 2, 4.

$A =$

A(1,1)	A(1,2)	A(1,3)	A(1,4)	A(1,5)
A(2,1)	A(2,2)	A(2,3)	A(2,4)	A(2,5)
A(3,1)	A(3,2)	A(3,3)	A(3,4)	A(3,5)
A(4,1)	A(4,2)	A(4,3)	A(4,4)	A(4,5)
A(5,1)	A(5,2)	A(5,3)	A(5,4)	A(5,5)

Vector Subscripts

- ▶ An integer array used as a subscript selects the listed elements, in any order and possibly more than once.
- ▶ Useful for picking entries by index, e.g. selected stars from a catalogue.
- ▶ A vector subscript can also be used on the left-hand side of an assignment, but then no index may repeat.

```
program vector_subscripts
  implicit none
  real :: x(6) = [10., 20., 30., 40., 50., 60.]
  integer :: idx(3) = [5, 1, 3]
  print *, x(idx)           ! 50. 10. 30.
  print *, x([2, 2, 6])     ! 20. 20. 60.
  x(idx) = 0.0              ! set the selected elements
  print *, x                 ! 0. 20. 0. 40. 0. 60.
end program vector_subscripts
```

Whole Arrays, Elements, and Sections

Declarations

```
real :: A(5), B(-1:3,3), C(5,3), D(5,3)
```

Whole arrays

```
A = 0.0      ! set every element of A to zero  
B = C + D    ! element-wise add C and D into B
```

Elements

```
A(1) = 0.0  
B(0,1) = A(3) + C(5,1)
```

Sections

```
A(2:4) = 0.0  
B(-1:0,1:2) = C(1:2,2:3) + 1.0
```

Whole-Array Expressions

Elemental semantics

Operators and elemental intrinsic functions (`sin`, `exp`, `abs`, ...) act element by element on conformable arrays.

Examples

```
real :: B(5,3), C(5,3), D(5,3)
B = C * D - B**2      ! element by element
B = sin(C) + cos(D)   ! elemental intrinsics
```

Not matrix multiplication

`C * D` multiplies matching elements. The matrix product is `matmul`, covered with the other array intrinsics below.

Array Assignment Evaluates the Right-Hand Side First

- ▶ In an array assignment, the whole right-hand side is evaluated before anything is stored.
- ▶ So `a(2:5) = a(1:4)` shifts the elements by one, as expected.
- ▶ A do loop that looks the same overwrites each element before it is read, and copies `a(1)` everywhere.

```
program shift
  implicit none
  integer :: i, a(5), b(5)
  a = [1, 2, 3, 4, 5]
  b = a
  a(2:5) = a(1:4)      ! array assignment
  print *, a           ! 1 1 2 3 4
  do i = 2, 5          ! the "same" as a loop
    b(i) = b(i-1)
  end do
  print *, b           ! 1 1 1 1 1
end program shift
```

Array Intrinsics: Inquiry and Reductions

- ▶ Inquiry: size, shape, lbound, ubound.
- ▶ Reductions: sum, product, minval, maxval, and the locations minloc, maxloc.
- ▶ With dim=, a reduction works along one dimension: sum(m, dim=1) sums each column.
- ▶ Logical reductions take a mask: count, any, all.

```
program intrinsics
  implicit none
  real :: m(2,3)
  m = reshape([1., 2., 3., 4., 5., 6.], [2, 3])
  ! m = | 1 3 5 |
  !     | 2 4 6 |

  print *, size(m), shape(m)           ! 6  2 3
  print *, lbound(m), ubound(m)        ! 1 1  2 3
  print *, sum(m), sum(m, dim=1)        ! 21.  3. 7. 11.
  print *, maxval(m), maxloc(m)         ! 6.  2 3
  print *, count(m > 3.0)               ! 3
  print *, any(m < 0.0), all(m > 0.0)   ! F T
end program intrinsics
```

Array Intrinsics: Matrices and Vectors

- ▶ `matmul(a, b)`: the matrix product.
The inner extents must agree:
 $[2, 3] \times [3, 2]$ gives $[2, 2]$.
- ▶ `dot_product(u, v)`: the scalar product of two vectors.
- ▶ `transpose(a)`: swaps rows and columns.
- ▶ `reshape(source, shape)`: fills a new shape in column-major order.

```
program linalg
  implicit none
  real :: a(2,3), b(3,2), c(2,2), u(3) = [1., 2., 3.]
  a = reshape([1., 2., 3., 4., 5., 6.], [2, 3])
  b = transpose(a)
  c = matmul(a, b)
  print *, dot_product(u, u)      ! 14.
  print *, c                      ! 35. 44. 44. 56.
end program linalg
```

Masked Assignment: where

- ▶ `where` assigns only to the elements where a logical mask is true. `elsewhere` handles the rest.
- ▶ Here, magnitudes are computed only for positive fluxes (`log10` is never called for the others), and the rest are flagged with 99.
- ▶ Short form for a single assignment:
`where (flux < 0.0) flux = 0.0`

```
program masked
  implicit none
  real :: flux(6) = [3.2, -1.0, 5.1, 0.0, -0.5, 2.4]
  real :: mag(6)
  where (flux > 0.0)
    mag = -2.5 * log10(flux)
  elsewhere
    mag = 99.0
  end where
  print *, mag      ! -1.26 99. -1.77 99. 99. -0.95
end program masked
```

Array I/O Ordering

```
print *, A
```

Writes all elements on one line, in column-major order: $A(1,1)$, $A(2,1)$, and so on.

```
read *, A
```

Reads the elements in the same order. A matrix stored row by row in a file must be read one row at a time, or transposed after reading.

Printing a matrix row by row

```
do i = 1, size(A, 1)
  print *, A(i,:)
end do
```

Array I/O Example

Program

```
program ex3
  implicit none
  integer, parameter :: n = 3
  integer :: a(n,n), i
  a = reshape([1, 2, 3, 4, 5, 6, 7, 8, 9], [n, n])
  print *, 'Element   =', a(3,2)
  print *, 'Column 1   =', a(:,1)
  print *, 'Subarray   =', a(:2,:2)
  print *, 'Whole      =', a
  print *, 'Transposed =', transpose(a)
  do i = 1, n
    print *, a(i,:)
  end do
end program ex3
```

Predict the output before you run it.

Array I/O Example

Program

```
program ex3
  implicit none
  integer, parameter :: n = 3
  integer :: a(n,n), i
  a = reshape([1, 2, 3, 4, 5, 6, 7, 8, 9], [n, n])
  print *, 'Element   =', a(3,2)
  print *, 'Column 1   =', a(:,1)
  print *, 'Subarray    =', a(2,:2)
  print *, 'Whole       =', a
  print *, 'Transposed =', transpose(a)
  do i = 1, n
    print *, a(i,:)
  end do
end program ex3
```

Predict the output before you run it.

Output (spacing condensed)

```
Element      = 6
Column 1     = 1 2 3
Subarray      = 1 2 4 5
Whole         = 1 2 3 4 5 6 7 8 9
Transposed   = 1 4 7 2 5 8 3 6 9
1 4 7
2 5 8
3 6 9
```

Catching Array Errors: -fcheck=all

- ▶ By default, Fortran does not check subscripts. Writing `x(6)` in a 5-element array silently overwrites other memory: the program may crash or, worse, run and give wrong results.
- ▶ Compile with `-g -fcheck=all` during development. The program then stops at the first bad subscript and says where. It also checks at run time that array shapes conform.
- ▶ The checks slow the program down, so leave them out of production runs.

```
program out_of_bounds
  implicit none
  integer :: i, n
  real :: x(5)
  n = 6
  do i = 1, n
    x(i) = i      ! x(6) does not exist
  end do
  print *, x
end program out_of_bounds
```

```
$ gfortran -g -fcheck=all out_of_bounds.f90
$ ./a.out
At line 7 of file out_of_bounds.f90
Fortran runtime error: Index '6' of dimension 1 of array '
  x' above upper bound of 5
```

Exercises

Exercise 1:

- ▶ Write a program that computes and prints the matrix multiplication of two real arrays.

$$A = \begin{pmatrix} 3 & 2 & 4 & 1 \\ 2 & 4 & 2 & 2 \\ 1 & 2 & 3 & 7 \end{pmatrix} \quad B = \begin{pmatrix} 3 & 2 & 4 \\ 2 & 4 & 2 \\ 1 & 2 & 3 \\ 0 & 2 & 1 \end{pmatrix}$$

Exercise 2:

- ▶ Write a Fortran program that reads an integer from the user and determines whether it is a palindrome (whether its digits read the same forwards and backwards).